

软件部分项目报告

信息与通信工程学院 徐子涵 大一 学号: 2025210236

Github 链接: https://github.com/LTH-XZH/group_homework_stm32

灯光控制器:

CubeMX 配置:

--开启 UART (使用 USART2) 的中断模式, PC6 作为 PWM 通道连接 LED。使用 TIM3 的 CH1 控制 LED。

--时钟设置: 分频器设为 84-1, 重装载器设置为 100-1, 使用 PWM 模式 1。

代码:

(1) 定义基本全局变量:

```
enum spectrum{on,off,blink,breath};
enum spectrum light_mode = off;//定义枚举变量存储灯光模式状态
char command[25];//字符数组存储UART接收到的命令
float frequency_blink;//闪烁模式频率
float frequency_breath;//呼吸模式频率
double brightness=128.00;//亮度
```

(2) 将命令解析放在中断处理函数中实现, 同时改变枚举变量的值

```
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size){
    if(huart==&huart2){
        HAL_UART_Transmit_IT(&huart2,(uint8_t*)command,Size);//返回给串口进行确认
        //----模式设定命令解析-----//
        //主要使用strncmp()函数,解析后更改枚举变量的值
        if(strncmp(command,"set_mode",8)==0){
            if(strncmp(command,"set_mode on",11)==0){
                light_mode = on;
            }
            else if(strncmp(command,"set_mode off",12)==0){
                light_mode = off;
            }
            else if(strncmp(command,"set_mode blink",14)==0){
                char fre_bl[4]="aaaa";//将设定的数字部分转移至一个新的字符数组中,方便转为浮点数
                for(int i=0;command[i+15]!='\0';i++){
                    fre_bl[i]=command[i+15];
                }
                frequency_blink=atof(fre_bl);//使用<stdlib.h>中的函数atof处理字符串转浮点数
                for(int i=0;i<4;i++){
                    fre_bl[i]='a';
                }
                //重新将fre_bl数组赋值为"aaaa",清空残留的数据,方便下次数字改变时重新转换
                light_mode = blink;
            }
        }
        //呼吸灯部分与闪烁模式逻辑相同
        else if(strncmp(command,"set_mode breathe",16)==0){
            char fre[4]="aaaa";
            for(int i=0;command[i+17]!='\0';i++){
                fre[i]=command[i+17];
            }
            frequency_breath=atof(fre);
            for(int i=0;i<4;i++){
                fre[i]='a';
            }
            light_mode = breath;
        }
    }
    //亮度设置部分,改变brightness的值
    else if(strncmp(command,"set_brightness",14)==0){
        char bri[5]="aaaaa";
        for(int i=0;command[i+15]!='\0';i++){
            bri[i]=command[i+15];
        }
        brightness=atof(bri);
        for(int i=0;i<5;i++){
            bri[i]='a';
        }
    }
    HAL_UARTEx_ReceiveToIdle_IT(&huart2,(uint8_t*)command,sizeof(command));//重新打开中断接收
}
```

(3) 在 while 死循环中使用 switch 语句判断灯光模式并进行设置

```
switch(light_mode){
    case on:
        light_on(brightness);
        break;
    case off:
        light_off();
        break;
    case blink:
        light_blink(frequency_blink);
        break;
    case breath:
        light_breath(frequency_breath);
        break;
}
```

(4) 定义各个模式的实现函数

```
//常量模式
void light_on(double x){
    __HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,(int)((100*x)/256));
}

//关闭
void light_off(){
    __HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,0);
}

//闪烁模式
void light_blink(float x){
    //通过循环实现闪烁
    while(1){
        __HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,99);
        HAL_Delay(500/x-1);
        __HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,0);
        HAL_Delay(500/x-1);
        if(light_mode!=blink||x!=frequency_blink) break;//退出条件，便于模式切换或者频率改变
    }
}

//呼吸实现和闪烁类似
void light_breath(float x){
    while(1){
        for(int i=0;i<100;i++){
            __HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,i);
            HAL_Delay(5/(2*x)-1);
        }
        for(int i=99;i>=0;i--){
            __HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,i);
            HAL_Delay(5/(2*x)-1);
        }
        if(light_mode!=breath||x!=frequency_breath) {
            __HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,50);
            break;
        }
    }
}
```

总结：经过实物连接验证，可以实现灯光控制要求。

测量——ADC：

此部分受限于时间和实践条件，手头缺乏相关可用于调试的设备。无法进行调试，也无法确定能否达到要求。

思路如下：

使用定时器产生每一秒一次的中断，在一秒内由 ADC 记录其中的上升沿或者下降沿次数，再通过计算得到方波信号的频率。使用 USART 发出。

PS：受限于对 ADC 的学习深入程度，目前无法实现较优的方案。计划此后更认真深入地学习相关部分。但将思路大致整理后也尝试写下了一部分代码。如定时器的中断回调函数，ADC 的 DMA 函数等。

电机部分：

受限于时间，学习进度还未达到，同时手头也缺乏相关可用于调试的设备，此部分任务目标暂时无法达到。也计划通过之后更加认真深入地学习来弥补。

补充说明：

1. 使用 keil 进行了代码编写，因为在这之前已经有配置过 keil 的环境，同时稍微熟悉过 keil 的使用流程。担心重新配置 Clion 的过程会和 keil 原有环境进行冲突造成各类麻烦耽误时间，故先继续使用。前文图片中呈现部分是为了方便添加注释从而更好地说明，使用 VS code 进行了编辑。
2. 碍于时间有限，并且是新接触一项不了解的事物，导致整体完成度不高。将通过更加专心投入、认真深入地学习来弥补所欠缺的部分。